

INTERFACE

Customize keyboard shortcuts

Customize keyboard shortcuts in Claude Code with a keybindings configuration file.

 Customizable keyboard shortcuts require Claude Code v2.1.18 or later. Check your version with `claude --version`.

Claude Code supports customizable keyboard shortcuts. Run `/keybindings` to create or open your configuration file at `~/.claude/keybindings.json`.

Configuration file

The keybindings configuration file is an object with a `bindings` array. Each block specifies a context and a map of keystrokes to actions.

 Changes to the keybindings file are automatically detected and applied without restarting Claude Code.

Field	Description
<code>\$schema</code>	Optional JSON Schema URL for editor autocompletion
<code>\$docs</code>	Optional documentation URL
<code>bindings</code>	Array of binding blocks by context

This example binds `Ctrl+E` to open an external editor in the chat context, and unbinds `Ctrl+U` :

```
{
  "$schema": "https://www.schemastore.org/claude-code-
keybindings.json",
  "$docs": "https://code.claude.com/docs/en/keybindings",
  "bindings": [
    {
      "context": "Chat",
      "bindings": {
        "ctrl+e": "chat:externalEditor",
        "ctrl+u": null
      }
    }
  ]
}
```

Contexts

Each binding block specifies a **context** where the bindings apply:

Context	Description
Global	Applies everywhere in the app
Chat	Main chat input area
Autocomplete	Autocomplete menu is open
Settings	Settings menu
Confirmation	Permission and confirmation dialogs
Tabs	Tab navigation components
Help	Help menu is visible
Transcript	Transcript viewer
HistorySearch	History search mode (Ctrl+R)
Task	Background task is running
ThemePicker	Theme picker dialog
Attachments	Image attachment navigation in select dialogs
Footer	Footer indicator navigation (tasks, teams, diff)
MessageSelector	Rewind and summarize dialog message selection

Context	Description
DiffDialog	Diff viewer navigation
ModelPicker	Model picker effort level
Select	Generic select/list components
Plugin	Plugin dialog (browse, discover, manage)
Scroll	Conversation scrolling and text selection in fullscreen mode
Doctor	/doctor diagnostics screen

Available actions

Actions follow a `namespace:action` format, such as `chat:submit` to send a message or `app:toggleTodos` to show the task list. Each context has specific actions available.

App actions

Actions available in the `Global` context:

Action	Default	Description
app:interrupt	Ctrl+C	Cancel current operation
app:exit	Ctrl+D	Exit Claude Code
app:redraw	(unbound)	Force terminal redraw
app:toggleTodos	Ctrl+T	Toggle task list visibility
app:toggleTranscript	Ctrl+O	Toggle verbose transcript

History actions

Actions for navigating command history:

Action	Default	Description
history:search	Ctrl+R	Open history search
history:previous	Up	Previous history item

Action	Default	Description
history:next	Down	Next history item

Chat actions

Actions available in the `Chat` context:

Action	Default	Description
chat:cancel	Escape	Cancel current input
chat:clearInput	Ctrl+L	Force a full screen redraw, preserving input. In fullscreen rendering , press twice within two seconds to run <code>/clear</code>
chat:clearScreen	Cmd+K	In fullscreen rendering , press twice within two seconds to run <code>/clear</code>
chat:killAgents	Ctrl+X Ctrl+K	Stop all running background subagents in this session
chat:cycleMode	Shift+Tab*	Cycle permission modes
chat:modelPicker	Meta+P	Open model picker
chat:fastMode	Meta+O	Toggle fast mode
chat:thinkingToggle	Meta+T	Toggle extended thinking
chat:submit	Enter	Submit message
chat:newline	Ctrl+J	Insert a newline without submitting
chat:undo	Ctrl+_, Ctrl+Shift+-	Undo last action
chat:externalEditor	Ctrl+G, Ctrl+X Ctrl+E	Open in external editor
chat:stash	Ctrl+S	Stash current prompt
chat:imagePaste	Ctrl+V (Alt+V on Windows and WSL)	Paste image from clipboard. On WSL, both shortcuts are bound by default

*On Windows without VT mode (Node <24.2.0/<22.17.0, Bun <1.2.23), defaults to Meta+M.

Autocomplete actions

Actions available in the `Autocomplete` context:

Action	Default	Description
autocomplete:accept	Tab	Accept suggestion
autocomplete:dismiss	Escape	Dismiss menu
autocomplete:previous	Up	Previous suggestion
autocomplete:next	Down	Next suggestion

Confirmation actions

Actions available in the Confirmation context:

Action	Default	Description
confirm:yes	Y, Enter	Confirm action
confirm:no	N, Escape	Decline action
confirm:previous	Up	Previous option
confirm:next	Down	Next option
confirm:nextField	Tab	Next field
confirm:previousField	(unbound)	Previous field
confirm:toggle	Space	Toggle selection
confirm:cycleMode	Shift+Tab	Cycle permission modes
confirm:toggleExplanation	Ctrl+E	Toggle permission explanation

Permission actions

Actions available in the Confirmation context for permission dialogs:

Action	Default	Description
permission:toggleDebug	(unbound)	Toggle permission debug info. The previous default of Ctrl+D was removed in v2.1.146 because it shadowed app:exit

Transcript actions

Actions available in the `Transcript` context:

Action	Default	Description
<code>transcript:toggleShowAll</code>	Ctrl+E	Toggle show all content
<code>transcript:exit</code>	q, Ctrl+C, Escape	Exit transcript view

History search actions

Actions available in the `HistorySearch` context:

Action	Default	Description
<code>historySearch:next</code>	Ctrl+R	Next match
<code>historySearch:accept</code>	Escape, Tab	Accept selection
<code>historySearch:cancel</code>	Ctrl+C	Cancel search
<code>historySearch:execute</code>	Enter	Execute selected command
<code>historySearch:cycleScope</code>	Ctrl+S	Cycle scope: session, project, everywhere

Task actions

Actions available in the `Task` context:

Action	Default	Description
<code>task:background</code>	Ctrl+B, Ctrl+X Ctrl+B	Background current task. The Ctrl+X Ctrl+B chord requires v2.1.169 or later and avoids the tmux prefix conflict

Theme actions

Actions available in the `ThemePicker` context:

Action	Default	Description
theme:toggleSyntaxHighlighting	Ctrl+T	Toggle syntax highlighting

Help actions

Actions available in the **Help** context:

Action	Default	Description
help:dismiss	Escape	Close help menu

Tabs actions

Actions available in the **Tabs** context:

Action	Default	Description
tabs:next	Tab, Right	Next tab
tabs:previous	Shift+Tab, Left	Previous tab

Attachments actions

Actions available in the **Attachments** context:

Action	Default	Description
attachments:next	Right	Next attachment
attachments:previous	Left	Previous attachment
attachments:remove	Backspace, Delete	Remove selected attachment
attachments:exit	Down, Escape	Exit attachment navigation

Footer actions

Actions available in the **Footer** context:

Action	Default	Description
footer:next	Right	Next footer item
footer:previous	Left	Previous footer item
footer:up	Up	Navigate up in footer (deselects at top)
footer:down	Down	Navigate down in footer
footer:openSelected	Enter	Open selected footer item
footer:clearSelection	Escape	Clear footer selection

Message selector actions

Actions available in the `MessageSelector` context:

Action	Default	Description
messageSelector:up	Up, K, Ctrl+P	Move up in list
messageSelector:down	Down, J, Ctrl+N	Move down in list
messageSelector:top	Ctrl+Up, Shift+Up, Meta+Up, Shift+K	Jump to top
messageSelector:bottom	Ctrl+Down, Shift+Down, Meta+Down, Shift+J	Jump to bottom
messageSelector:select	Enter	Select message

Diff actions

Actions available in the `DiffDialog` context:

Action	Default	Description
diff:dismiss	Escape	Close diff viewer
diff:previousSource	Left	Previous diff source
diff:nextSource	Right	Next diff source
diff:previousFile	Up, K	Previous file in the file list; scroll up one line in the detail view
diff:nextFile	Down, J	Next file in the file list; scroll down one line in the detail view
diff:viewDetails	Enter	View diff details

Action	Default	Description
diff:back	(context-specific)	Go back in diff viewer

The diff detail view also binds pager-style keys to the standard scroll actions. These bindings are part of the `DiffDialog` context and apply only in the detail view; the `Scroll` context defaults listed under Scroll actions are unchanged.

Action	Default	Description
scroll:pageUp	PageUp	Scroll up half a viewport
scroll:pageDown	PageDown	Scroll down half a viewport
scroll:fullPageUp	Shift+Space, B	Scroll up a full viewport
scroll:fullPageDown	Space	Scroll down a full viewport
scroll:top	G, Home	Jump to the top
scroll:bottom	Shift+G, End	Jump to the bottom

Model picker actions

Actions available in the `ModelPicker` context:

Action	Default	Description
modelPicker:decreaseEffort	Left	Decrease effort level
modelPicker:increaseEffort	Right	Increase effort level
modelPicker:thisSessionOnly	s	Apply highlighted model to this session only

Select actions

Actions available in the `Select` context:

Action	Default	Description
select:next	Down, J, Ctrl+N	Next option
select:previous	Up, K, Ctrl+P	Previous option

Action	Default	Description
select:accept	Enter	Accept selection
select:cancel	Escape	Cancel selection

Plugin actions

Actions available in the `Plugin` context:

Action	Default	Description
plugin:toggle	Space	Toggle plugin selection
plugin:install	I	Install selected plugins
plugin:favorite	F	Favorite the selected plugin so it sorts near the top of the Installed tab

Settings actions

Actions available in the `Settings` context. The `select:accept` and `confirm:no` actions are reused from the Select and Confirmation contexts with Settings-specific behavior: changes apply to each setting as soon as you change it, so Escape closes the panel with your changes saved rather than declining.

Action	Default	Description
settings:search	/	Enter search mode
settings:retry	R	Retry loading usage data on error
select:accept	Enter, Space	Change the selected setting or open its submenu
confirm:no	Escape	Close the panel. Changes are already saved

Doctor actions

Actions available in the `Doctor` context:

Action	Default	Description
<code>doctor:fix</code>	F	Send the diagnostics report to Claude to fix the reported issues. Only active when issues are found

Voice actions

Actions available in the `Chat` context when voice dictation is enabled:

Action	Default	Description
<code>voice:pushToTalk</code>	Space	Dictate a prompt. Hold or tap depending on <code>/voice</code> mode

Scroll actions

Actions available in the `Scroll` context when fullscreen rendering is enabled:

Action	Default	Description
<code>scroll:lineUp</code>	(unbound)	Scroll up one line. Mouse wheel scrolling triggers this action
<code>scroll:lineDown</code>	(unbound)	Scroll down one line. Mouse wheel scrolling triggers this action
<code>scroll:pageUp</code>	PageUp	Scroll up half the viewport height
<code>scroll:pageDown</code>	PageDown	Scroll down half the viewport height
<code>scroll:top</code>	Ctrl+Home	Jump to the start of the conversation
<code>scroll:bottom</code>	Ctrl+End	Jump to the latest message and re-enable auto-follow
<code>scroll:halfPageUp</code>	(unbound)	Scroll up half the viewport height. Same behavior as <code>scroll:pageUp</code> , provided for vi-style rebinds
<code>scroll:halfPageDown</code>	(unbound)	Scroll down half the viewport height. Same behavior as <code>scroll:pageDown</code> , provided for vi-style rebinds
<code>scroll:fullPageUp</code>	(unbound)	Scroll up the full viewport height
<code>scroll:fullPageDown</code>	(unbound)	Scroll down the full viewport height
<code>selection:copy</code>	Ctrl+Shift+C / Cmd+C	Copy the selected text to the clipboard
<code>selection:clear</code>	(unbound)	Clear the active text selection
<code>selection:extendLeft</code>	Shift+Left	Extend the active selection one column left

Action	Default	Description
<code>selection:extendRight</code>	Shift+Right	Extend the active selection one column right
<code>selection:extendUp</code>	Shift+Up	Extend the active selection one row up. Scrolls the viewport when the selection reaches the top edge
<code>selection:extendDown</code>	Shift+Down	Extend the active selection one row down. Scrolls the viewport when the selection reaches the bottom edge
<code>selection:extendLineStart</code>	Shift+Home	Extend the active selection to the start of the line
<code>selection:extendLineEnd</code>	Shift+End	Extend the active selection to the end of the line

Keystroke syntax

Modifiers

Use modifier keys with the `+` separator:

- `ctrl` or `control` - Control key
- `shift` - Shift key
- `alt` , `opt` , `option` , or `meta` - Alt key on Windows and Linux, Option key on macOS
- `cmd` , `command` , `super` , or `win` - Command key on macOS, Windows key on Windows, Super key on Linux

The `cmd` group is only detected in terminals that report the Super modifier, such as those supporting the Kitty keyboard protocol or xterm's `modifyOtherKeys` mode. Most terminals do not send it, so use `ctrl` or `meta` for bindings you want to work everywhere.

For example:

<code>ctrl+k</code>	Ctrl + K
<code>shift+tab</code>	Shift + Tab
<code>meta+p</code>	Option + P on macOS, Alt + P elsewhere
<code>ctrl+shift+c</code>	Multiple modifiers

Uppercase letters

A standalone uppercase letter implies Shift. For example, `K` is equivalent to `shift+k` . This is useful

for vim-style bindings where uppercase and lowercase keys have different meanings.

Uppercase letters with modifiers (e.g., `ctrl+K`) are treated as stylistic and do **not** imply Shift:

`ctrl+K` is the same as `ctrl+k` .

Chords

Chords are sequences of keystrokes separated by spaces:

```
ctrl+k ctrl+s    Press Ctrl+K, release, then Ctrl+S
```

Special keys

- `escape` or `esc` - Escape key
- `enter` or `return` - Enter key
- `tab` - Tab key
- `space` - Space bar
- `up` , `down` , `left` , `right` - Arrow keys
- `backspace` , `delete` - Delete keys

Unbind default shortcuts

Set an action to `null` to unbind a default shortcut:

```
{
  "bindings": [
    {
      "context": "Chat",
      "bindings": {
        "ctrl+s": null
      }
    }
  ]
}
```

This also works for chord bindings. Unbinding every chord that shares a prefix frees that prefix for

use as a single-key binding:

```
{
  "bindings": [
    {
      "context": "Chat",
      "bindings": {
        "ctrl+x ctrl+k": null,
        "ctrl+x ctrl+e": null,
        "ctrl+x": "chat:newline"
      }
    }
  ]
}
```

If you unbind some but not all chords on a prefix, pressing the prefix still enters chord-wait mode for the remaining bindings.

Reserved shortcuts

These shortcuts cannot be rebound:

Shortcut	Reason
Ctrl+C	Hardcoded interrupt/cancel
Ctrl+D	Hardcoded exit
Ctrl+M	Identical to Enter in terminals (both send CR)
Caps Lock	Not delivered to terminal applications

Terminal conflicts

Some shortcuts may conflict with terminal multiplexers:

Shortcut	Conflict
Ctrl+B	tmux prefix (press twice to send)
Ctrl+A	GNU screen prefix

Shortcut	Conflict
Ctrl+Z	Unix process suspend (SIGTSTP)

Vim mode interaction

When vim mode is enabled via `/config` → Editor mode, keybindings and vim mode operate independently:

- **Vim mode** handles input at the text input level (cursor movement, modes, motions)
- **Keybindings** handle actions at the component level (toggle todos, submit, etc.)
- The Escape key in vim mode switches INSERT to NORMAL mode; it does not trigger `chat:cancel`
- Most Ctrl+key shortcuts pass through vim mode to the keybinding system
- In vim NORMAL mode, `?` shows the help menu (vim behavior)
- In vim NORMAL mode, `/` opens history search, the same as Ctrl+R in standard mode

Validation

Claude Code validates your keybindings and shows warnings for:

- Parse errors (invalid JSON or structure)
- Invalid context names
- Reserved shortcut conflicts
- Terminal multiplexer conflicts
- Duplicate bindings in the same context

Run `/doctor` to see any keybinding warnings.